current log gets too big. To use the rotating handler, you must specify an additional argument of `maxBytes` that specifies the file's size limit.

There are plenty of sources to dig into for more information on logging:

- *Django's logging documentation*—https://docs.djangoproject.com/en/dev/topics/logging/
- *Python's logging documentation*—https://docs.python.org/3/library/logging.html
- *Python's logging cookbook*—https://docs.python.org/3/howto/logging-cookbook.html

If all that isn't enough, there are also centralized log-management services. These providers are API based, and you send your log messages to their systems. They typically include log-analysis tools and allow you to have all of your servers log to the same place, giving you an overview of your entire deployment. Two popular log management services are Datadog and Sematext, and the cloud providers have centralized logging within their offerings as well.

B.4.4 Custom 404 and 500 pages

The Django development server provides a lot of handy information when something goes wrong. The `404` page displays which URLs were attempted, and the `500` page shows a complete stack trace. Neither of these are things you want your users to see. With `DEBUG=False`, this information isn't shown, but the default pages won't look like your site. This is simple to remedy. Create files named *404.html* and *500.html* inside your templates folder, and Django will automatically use them to display the corresponding errors. These files are Django templates, so you can extend your base template and have the errors look like your project's other pages.

If this isn't sufficient for your error-handling needs, Django allows you to override the views called when errors have been triggered. You can replace them with your own views and perform whatever actions you want. For details on how to write custom error views, see https://docs.djangoproject.com/en/dev/topics/http/views/#customizing-error-views.

B.5 Static file management

In chapter 7, you learned how to use static files, such as CSS and JavaScript, within your Django project. The two key configuration values in settings.py for static files are the following:

```
# RiffMates/RiffMates/settings.py
...
STATIC_URL = '/static/'
STATICFILES_DIRS = (
    BASE_DIR / 'static',
)
```

The base URL for serving static files

The location where static files are found in the project